Nuclear Materials and Energy 49 (2026) 102250

Contents lists available at ScienceDirect

Nuclear Materials and Energy

journal homepage: www.elsevier.com/locate/nme

Presentation of the open-source distribution of the SOLPS-ITER plasma edge modelling suite☆

X. Bonnin [a,*], O. Hoenen [a], S.D. Pinches [a], A.A. Pshenov [a], W. Dekeyser [b], W.Van Uytven [b], E. Kaveeva [c], N. Shtyrkhunov [c], A. Zito [d]

[a] ITER Organization, Route de Vinon-sur-Verdon, CS 90 046, 13067 St-Paul-lez-Durance, France
[b] Department of Mechanical Engineering, KU Leuven, Leuven, Belgium
[c] Peter the Great St. Petersburg Polytechnic University, St. Petersburg, Russian Federation
[d] Max-Planck-Institut für Plasmaphysik, Garching, Germany

A B S T R A C T

The SOLPS-ITER code suite is one of the main tools used worldwide for tokamak edge modelling. Launched in 2015 and maintained and distributed by the ITER Organization (IO), it was recently extended to a "Wide Grids" version, allowing the plasma solver computational domain to extend to the full vacuum vessel and provide a much more detailed description of the interactions between the Scape-Off Layer (SOL) plasma and the first wall. It remains the main workhorse for ITER design and operation planning activities, in addition to its use in the ITER Member States for simulating existing experiments and predicting future device performance.

To accompany the development of the private fusion ventures landscape, the IO has triggered a large effort to make as many as possible of its software and knowledge sources publicly available. Among these is SOLPS-ITER. It is thus now available as open-source software, on GitHub, as releases 3.0.10(a), 3.1.2(a), and 3.2.1(a), along with its underlying components and several library dependencies that were not previously available. These include the B2.5 plasma fluid solver, the Eirene Monte-Carlo neutral kinetic solver, the Carre grid generator, the Uinp case build-up tool, and many analysis and post-processing visualization tools, along with the support libraries GR for graphics and MSCL for input data file handling.

This paper will provide details on what features are available as part of the open-source distribution, how to make the best use of them and how to contribute and participate in further development of the code.

General presentation of the SOLPS-ITER code suite

The SOLPS-ITER code suite [1,2] is a tool primarily devoted to modelling tokamak edge plasmas, covering a domain from the outer edge of the confined region to the scrape-off layer (SOL), divertor, and interface with the vessel walls [3]. Over the years, it has been expanded to also be applicable to linear plasma devices [4] and partially ionized plasmas [5], insofar as the underlying assumptions of its physics model can be met. Its main engine is the **B2.5**[1] multi-fluid plasma transport code, relying on the Braginskii equations, and for which a very complete description of the physics model can be found in [6]. Neutrals can be treated either as additional fluids or kinetically, in which case **B2.5** is coupled to the **Eirene** code [7], providing a Monte-Carlo description of neutral trajectories and neutral-plasma interactions. This **B2.5-Eirene** coupling is often what is referred to as SOLPS (Scrape-Off Layer Plasma Simulator), but the full code suite also contains the **Carre** grid generator [8], the **DivGeo** graphical interface for specifying the case geometry and wall state, the **Uinp** tool to build the input files, the **b2plot** graphical utility to visualize the results, and a large collection of ancillary utility programs, processing and analysis scripts to make SOLPS-ITER a fully contained and versatile toolbox.

Since the original launch of SOLPS-ITER in 2015 [2] by the ITER Organization, building on the SOLPS5.3 kernel [1], the code suite has grown with the addition of the **SOLPS-GUI** [9] for setting up and managing cases, and has been adapted to the IMAS (Integrated Modelling and Analysis Suite [10]) framework. The IMAS framework is the overarching infrastructure used for data representation, manipulation, storage, and modelling at ITER, and being adopted across the ITER Member States as the industry's standard. The data is represented in self-contained IDSs (Interface Data Structures), each devoted to a particular

☆ This article is part of a special issue entitled: 'PSI-27 Proceedings' published in Nuclear Materials and Energy.

* Corresponding author.
E-mail address: xavier.bonnin@iter.org (X. Bonnin).

[1] For consistency with the SOLPS-ITER documentation, we adopt in this paper the same typography: code components are listed in **bold**, while individual files and variables are in *italics*.

https://doi.org/10.1016/j.nme.2026.102250
Received 16 July 2026; Received in revised form 29 September 2026; Accepted 29 October 2026
Available online 4 November 2026
2352-1791/© 2026 Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

aspect, following rules specified in the IMAS Data Dictionary, and manipulated using a data Access Layer library with interfaces available for most commonly-used programming languages in the community.

In SOLPS-ITER's instance, this means that a case can be started by having **DivGeo** import both an *equilibrium* and a *wall* IDS from IMAS, and the SOLPS-ITER output can be written out also as IDSs, namely *edge_profiles*, *edge_transport*, *edge_sources*, *radiation*, *summary*, and *wall*, and provide a SimDB (Simulation DataBase manager) manifest for ingestion as a data entry in a machine simulations database. SOLPS-ITER is compatible with IMAS Data Dictionary versions 3 and 4, and uses IMAS Access Layer 5, but, to get use of all the features, it is recommended to link to the most recent IMAS software possible. The compatibility of the code base with ongoing IMAS developments is actively maintained.

In order to facilitate code usage, not only among public and academic institutions within the ITER Member States, but also private fusion enterprises and third-party country actors, the SOLPS-ITER code suite has been made publicly available on GitHub, at https://github.com/iterorganization/SOLPS-ITER. This lifts the requirement for users to hold an ITER computing account to get access to the code. The main SOLPS-ITER repository contains the top layer of the code, its documentation, the scripts and programs needed to prepare cases, run the code and analyze results, but the main code components (**B2.5**, **Eirene**, **Carre**, and **DivGeo**), as well as a few support libraries, are found as separate modules in the same *iterorganization* GitHub domain. The GitHub page also includes a Wiki section, which provides a full library of papers describing the evolution of the code and some of its most important features, along with tips to help users with common problems and minutes of past User Forum meetings (held about every six weeks to discuss code issues with the community).

Files too large to be stored on GitHub, such as wall reflection datasets and example cases, are stored as Zenodo records at https://zenodo.org/communities/solps/. Although some additional proprietary datasets may be used when performing calculations, these are not required by the standard setup, which relies exclusively on public-domain accessible datasets. In particular, this means that, for the ADAS atomic physics package, the code installation script will fetch data from the Open-ADAS website if the user does not have access to the ITER ADAS database. The Zenodo location is also available for authors of SOLPS-related papers to share their simulation data, so as to fulfill their data availability obligations.

The overall code and the **B2.5** plasma fluid solver are distributed under an EUPL licence. Eirene has been made available by Forschungszentrum Jülich using a CC_BY-NC-ND license for general usage without modification of the code, but code developers have to sign on to additional licensing terms, ensuring a proper propagation and integration of any new code additions. The other code components are distributed under an LGPL version 3 license. Commercial use of the code is allowed. Further distribution of code modifications may occur only under the terms of the same or a compatible license. Contributors are strongly encouraged to make their modifications publicly available as forks of the ITER Organization GitHub server.

The code is distributed in three main "flavours", corresponding to version numbers 3.0.x, 3.1.x, and 3.2.x. In this paper, we will be referring to the most recent releases for each version at the time of submission (September 2026), namely versions 3.0.10a, 3.1.2a, and 3.2.1a. The reference, traditional version, here 3.0.10a, relies on a 5-point stencil to solve the basic plasma transport equations, extended to 7-, 9-, or 11-point when accounting for certain drift terms,[2] and its computational domain can only extend up to the last flux surface that goes from one divertor plate to another without intersecting any vacuum vessel wall structures. The grid cells are quadrangular with two field-aligned sides. Such a grid is often referred to as a "structured" and/or "narrow" grid, and maps to a rectangular computational domain. This reference version is provided either from the code's *master* or *develop* repository branches. The *master* branch is updated only a few times a year at most, and is recommended for users wanting a stable code base which to use for large sets of production runs. The *develop* branch represents the leading edge of current code advances and will contain the latest bug fixes and newest features.

The 3.1.x version differs from 3.0.x in that the standard solution stencil is 9-point based, but otherwise the physics model is essentially the same, except that additional flux components may exist. Its use is recommended when modelling fluid neutrals, to obtain a correct picture of their flows if the grid is distorted to match the shape of the divertor targets [11]. It is available as the *release/3.1.2* branch. Its grid is also "structured" and "narrow".

The 3.2.x version, found in the *feature/wg-release* branch, expands the possible solution domain all the way to the vacuum vessel walls, and introduces the possibility of non-quadrangular grid cells [3]. The computational domain can have an arbitrary shape. As such, it is often referred to as the "unstructured" and/or "wide (or extended) grids" code version. It represents a complete overhaul of the original code base, including new data structures used to represent the plasma state and derived quantities and a new version of the grid generator, **Carre2**. This "wide grids" (WG) version is intended, in the long run, to become the reference SOLPS-ITER version, at which point the structured code versions will no longer be actively maintained. To reach this horizon, however, requires the WG code to contain all the physics and numerics capabilities of the structured code, which is not quite yet the case at time of publication, as will be detailed further below.

The SOLPS-ITER literature sometimes uses interchangeably the "structured/unstructured" and "narrow/wide grids" labels. To be clear, a "structured" grid is a grid composed exclusively of quadrangular cells, and which maps to a rectangular computational domain, where each cell can be uniquely identified by a set of two indices, one in the radial and one in the poloidal direction. An "unstructured" grid can contain cells of arbitrary polygonal shape, and the cells are mapped to a one-dimensional list. A "narrow" grid is one in which the computational domain for the **B2.5** plasma solver stops at the last flux surface that starts and ends at a divertor target. A "wide" grid will instead extend to contact the whole first wall contour. The 3.0.x and 3.1.x code versions can only deal with structured narrow grids, while the 3.2.x code versions are designed to handle both structured narrow and unstructured, narrow or wide, grids.

To each of the code versions mentioned above, correspond branches in the **B2.5**, **Eirene**, **Carre(2)**, and **DivGeo** GitHub submodules, as well as scripts to switch from one version to the other, as described in Table 1.

[2] By 5-point stencil, we mean that the matrix elements used to solve the discretized equations only involve the interaction between a cell and its immediate top, bottom, left, and right neighbours. To solve for the perpendicular viscosity-driven current, which involves a 4th-order radial derivative, the second nearest top and bottom neighbours must also be considered, expanding the stencil to 7-point. The 9-point stencil refers to when the top-left, top-right, bottom-left, and bottom-right neighbours are included in the discretization, as is needed for instance when treating $\mathbf{E}\times\mathbf{B}$, grad-B and diamagnetic drift terms. The 11-point stencil is the 9-point stencil with the second-nearest top and bottom neighbours added.

Table 1
Description of the various main SOLPS-ITER code distribution versions and their submodule branches. The table also shows the script available to automatically update that branch to its current state on the repository.

Code version	3.0.10a master	3.0.10a develop	3.1.2a (9-point-stencil)	3.2.1a (wide grids)
Update script	*solps-iter_update*	*solps-iter_update_develop*	*solps-iter_update_3.1.2_release*	*solps-iter_update_wg_release*
SOLPS-ITER branch name	*master*	*develop*	*release/3.1.2*	*feature/wg_release*
B2.5 branch name	*master*	*develop*	*release/3.1.2*	*feature/wg_release*
Eirene branch name	*SOLPS_master*	*SOLPS_develop*	*SOLPS_develop*	*SOLPS_develop*
Carre branch name	*master*	*develop*	*develop*	develop
Carre2 branch name	N/A	N/A	N/A	*feature/Carre2_wg_release*
DivGeo branch name	*master*	*develop*	*develop* or *release/3.1.2*	*feature/wg_release*
ADAS branch name	*master*	*develop*	*develop*	*develop*

When using multiple code versions, it is however recommended to have each code version stored in a separate tree to avoid confusion.

SOLPS-ITER physics model and feature extensions

The transition of SOLPS-ITER to an open-source distribution inaugurates the new set of code versions, 3.0.10, 3.1.2, and 3.2.1, as described above. Along with the changes in infrastructure necessary for the deployment on GitHub, these new versions also contain improvements to the physics model and code capabilities available to users. First, the code releases have been made compatible with IMAS Data Dictionary versions up to 4.1.1 and Access Layer versions up to 5.7.1, while maintaining full backward compatibility. The physical constants have been updated to use the CODATA 2022 standard. An important new option is the output and archival of full cases as SimDB data entries, containing a full input/output file manifest with which it is possible to restart a case. The SOLPS-ITER workflow has been adapted to support the Eirene input file provided either in the new JSON (JavaScript Object Notation) format[3] or in the historical fixed ASCII format. A significant effort has been devoted to improve the code numerics when dealing with time-dependent problems [12–14], most notably for the wide grids code version, in which both the Crank-Nicolson and BDF2 time differentiation schemes have been implemented in addition to the original Euler scheme [15]. New neoclassical transport corrections are available for dealing with heavy impurities [16]. At the sheath entrance, non-marginal velocity boundary conditions can now be imposed, in a drift-compatible way, which, for multi-species plasmas, no longer force heavy impurities to be accelerated to a velocity close to the hydrogenic species sound speed [17], making the model more realistic and avoiding exaggerated sputtering yields from said heavy impurities. Additional output capabilities, such as sputtering rates per incident species, and new post-run analysis scripts, are also part of the updated code suite package.

Code quality is ensured by a large set of continuous integration (CI) and regression tests. The tests that must be passed with each new code commit include:

- Compilation of each code component individually and of the coupled **B2.5-Eirene** code, with/without MPI, OpenMP, and debug options
- A **B2.5** standalone run (including its pre-processing programs) with full debug options turned on, compared to a reference
- A reproducible coupled **B2.5-Eirene** case, compared to a reference
- A coupled case designed to check that both **Eirene** input methods (JSON and ASCII) yield the same results
- A restart effect test, comparing a 20-timestep run to a 10-timestep run restarted for another 10 timesteps
- A reproducible coupled run using MPI parallelization, compared to a reference
- A **B2.5** stand-alone run using OpenMP parallelization, compared to a reference
- A reproducible coupled run using OpenMP parallelization, compared to a reference
- A build of the SOLPS-ITER and **Eirene** manuals from their LaTeX base.

For the purposes of the CI tests, a reproducible coupled run means one where the **Eirene** settings are such that the random number seeds are kept the same and correlated sampling is used, so that the Monte-Carlo results, on an identical plasma background, will also be identical. The chosen test cases are also selected to cover various geometries (linear devices, single-null, and double-null topologies) and species mixes (single-fluid, with impurities, with sputtering (both chemical and physical), and Helium-dominated plasmas). The CI test scripts are part of the standard code distribution and the reference cases for these tests are included as part of the standard set of sample cases on Zenodo so that prospective developers can run the CI tests in their own environment first to make sure they pass before committing any new code features. Once a Pull Request is made, the CI tests, which are run on the ITER computer infrastructure, are automatically triggered and must pass for a code merge to be approved. Of course, in some cases, the proposed code change will affect, in an expected way, the reference cases, and the submitting developer can then also provide a new Zenodo record containing the updated reference cases with proper documentation.

Dedicated Wide grids developments

The Wide Grids implementation constitutes the largest ongoing development effort within SOLPS-ITER. The approach introduces new and arbitrary mesh topologies, revised plasma-state data structures and generalized numerical operators suitable for non-structured computational domains. Because the initial WG code development forked away from the original structured code (version 3.0.6) back in 2016, a lot of subsequent development in the 3.0.x line had to be re-instated, whenever relevant and desirable, in the unstructured code. This work is not yet complete, but most features that users routinely request are now available. Further feature inclusions will be driven by actual use cases when a structured code option is found necessary in the unstructured code for a particular physics problem being studied. Examples of such code features ported from recent structured code improvements include the implementation of the full Zhdanov closure formalism [18–20], H-mode-like transport coefficient profiles, advanced feedback controllers, and the **b2plot** graphical post-processor.

Some other code work was only done in the Wide Grids code, since it is intended to become the reference code version and thus no backward-compatibility with the older structured code is supported, such as the Crank-Nicolson and BDF2 time integration schemes already mentioned above, collisionality-based flux limiters, or an option to use the PARDISO matrix solver instead of the original MA28 one. The Advanced Fluid Neutrals (AFN) model present in the Wide Grids code implementation has also been considerably expanded [21].

In addition, the move to a fully unstructured grid scheme requires some precautions in numerically challenging regions to be taken in order to improve robustness while preserving physically meaningful

[3] Available at https://github.com/jacobwilliams/json-fortran .

transport behaviour. Three such cases are of particular importance: "shaved flux tubes", "wall-tangent flux tubes", and strongly distorted grids in the poloidal direction.

Shaved flux tubes

A common occurrence in full vessel field-aligned grids is the "shaved" flux tube, where the intersection of the end of flux tube with the wall happens over several cells in the poloidal direction, if the angle of incidence of the flux tube onto the wall is shallow enough. In such cases, if a parallel temperature gradient is established along the last flux tube cells, this will induce a short-circuiting thermo-electric current between neighbouring cells and lead to strong electric potential equation instabilities. To avoid this issue, two mitigating strategies can be employed (see Fig. 1). At the grid generation step, the cells can be rearranged, from a set of trapezoids ending with a triangle, into a set of triangles, all sharing one common vertex (the intersection point of the outer flux tube surface with the wall) and stacked to reach the successive grid points along the inner flux tube surface, such that only one triangle has a face shared with the wall contour. If this is not or cannot be done, then the solution applied is to artificially increase the poloidal transport coefficients between the shaved flux tube cells so as to effectively create a single "super-cell" in which the plasma parameters will be essentially constant and thus damp out any spurious currents.

Wall-tangent flux tubes

To deal with areas where the flux tubes are running along the vessel wall's contour, the sheath boundary condition needed to be modified. First, it now includes a diffusive component to reflect the radial plasma leakage [3]. Second, a minimal impact angle is imposed between the wall and the local magnetic field direction, so that some poloidal flow can be maintained. At the place where the incidence angle is shallowest, the imposed impact angle is made to flip sign, dividing the flux tube into upstream-pointing and downstream-pointing halves.

Poloidally distorted grids

A remaining issue that is still, at publication time, being worked on, is the numerical instability shown by the unstructured grid solver when dealing with strongly distorted grids, such as the ones commonly found in older code version runs when gridding cases where the target surface and flux tubes are not nearly orthogonal to each other. Indeed, if one, for instance, attempts to take a standard ITER case from the structured code database (built with version 3.0.x or even older SOLPS implementations) and continue it with version 3.2.1(a), using the same grid and starting from the existing converged solution, the case very quickly crashes. The issue has been identified as due to the lack of slope-limiting measures and other such stabilization mechanisms in the currently implemented solver, and for which various solutions are being investigated, to be reported on in a future publication.

Anomalous radial ion heat fluxes

Another important development concerns the formulation of ion heat fluxes in the Wide Grids version. It is now possible to switch between the adiabatic and isothermal models for the ion heat flux associated with anomalous transport, by changing the prefactor multiplier coefficient when computing the particle fluxes that carry heat. This prefactor is noted $f_{(he)}$ in Eqs. (1–2), respectively for the poloidal and radial ion particle fluxes, following the notation from [6]. Here, the x and y indices correspond to the poloidal and radial directions, respectively, a is the species index, $V^{(E)}$ refers to the $\mathbf{E} \times \mathbf{B}$ drift velocity, $V^{(AN)}$ to the velocity driven by anomalous transport, V^{cor} to the velocity driven the Coriolis force. $D_{n,a}$ is the density (n_a) gradient-driven particle diffusivity, and $D_{p,a}$ is the pressure (p_a) gradient-driven particle diffusivity, while b_x is the poloidal component of the unit vector in the direction of the magnetic field.

$$\Gamma_{ax}^{(he)} = \left(b_x V_{a\|} + V_{ax}^{(E)} + f_{(he)} V_a^{(AN)} + V_a^{cor}\right) n_a - f_{(he)} D_{n,a} \frac{1}{h_x} \frac{\partial n_a}{\partial x} - \frac{5}{3} D_{p,ax} \frac{1}{h_x} \frac{\partial p_a}{\partial x} \tag{1}$$

$$\Gamma_{ay}^{(he)} = \left(f_{(he)} V_{ay}^{(AN)} + V_{ay}^{(E)}\right) n_a - \left[f_{(he)} D_{n,a} \frac{1}{h_y} \frac{\partial n_a}{\partial y} + \frac{5}{3} D_{p,ay} \frac{1}{h_y} \frac{\partial p_a}{\partial y}\right] \tag{2}$$

Historically, in the SOLPS5.2 physics model, $f_{(he)}$ held a value of 5/3, i.e. an adiabatic model was applied. However, in the Wide Grids code, its default value is now 1 (3/3), corresponding to an isothermal description. This change in default settings was done for three reasons. First, it matches the underlying physics assumptions of the k-ε model, another significant extension of the Wide Grids code capabilities [22], allowing for a physics-based determination of the anomalous transport. Second, it corresponds better to the current understanding of turbulent transport in the far SOL, dominated by the radial $\mathbf{E} \times \mathbf{B}$ drift motion of plasma filaments born at the separatrix from turbulent activity [23]. Lastly, from a purely phenomenological point of view, it also prevents an unphysical rise of the ion temperature in the far SOL that was noticed in some ITER cases.

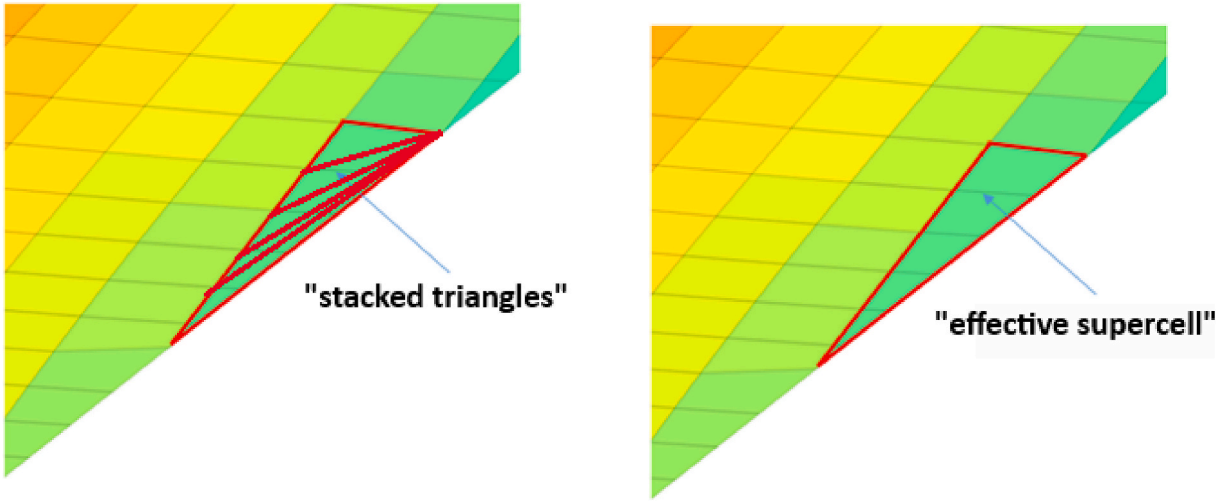


Fig. 1. A shaved flux tube end, resolved either with the stacked triangles method (left) or treated as an effective supercell (right).

Code and data availability

To obtain the code, it suffices to clone its repository from the ITER Organization GitHub website at https://github.com/iterorganization/SOLPS-ITER, and follow the procedure provided in the documentation for download and installation instructions, which will then fetch all the other needed code components. If one is interested in a single code component, these can also be cloned individually, and are found at the same https://github.com/iterorganization location. Currently, the distributed releases are for the 3.0.10a, 3.1.2a, and 3.2.1a code versions.

Among the case examples available on Zenodo, one will find, for the structured codes:

- *test_slab_ortho_standalone*: an orthogonal slab test case for **B2.5** standalone
- *AUG_16151_D*: ASDEX-Upgrade D-only benchmark cases, including **B2.5** standalone, coupled and single **Eirene** call run directories
- *AUG_16151_D+C+He*: ASDEX-Upgrade multi-fluid benchmark cases, including **B2.5** standalone, coupled and single **Eirene** call run directories
- *AUG_28903_D+N_drifts*: an ASDEX-Upgrade D + N case including drifts
- *AUG_28903_D+Ne_drifts*: an ASDEX-Upgrade D + Ne case including drifts
- *DIIID_3106_DN_D+C*: a DIII-D disconnected double-null case including Carbon chemical sputtering
- *ITER_2171_D+He+Be+Ne*: an ITER full performance case with Neon injection and Beryllium sputtering
- *ITER_2264_0.6Ne_100MW*: another ITER full performance case with Neon injection
- *ITER_2275_Donly_20MW*: an ITER low power case at low density (D-fuelled)
- *ITER_2292_Honly_20MW* to *ITER_2308_Honly_20MW*: a throughput scan of ITER low power cases (H-fuelled), spanning from the conduction-limited regime to full divertor detachment
- *ITER_2360_0.95He+0.05H_40MW*: an ITER Helium plasma case at mid-power (also contains an example with metastable-resolved Helium neutrals)
- *ITER_2588_D+He+N*: an ITER full power case with Nitrogen injection and gas puff feedback on SOL D particle content
- *ITER_2588_drifts*: an ITER **B2.5** standalone D-only demonstration case with and without drifts
- *ITER_535_D+He+Ar*: an ITER full power case with Argon injection
- *ITER_Be-W_D+T+He+Ne*: an ITER full power case with 98 species, fully resolving tungsten (W) species, and including the Zhdanov closure
- *JTEXT_Limiter_H*: a J-TEXT limiter case (H-only, full drifts, 5-point stencil only)
- *MAST_U_SFminus_Donly*: a lower snowflake-minus case from MAST-U at low power, D-only

as well as additional partial cases used in tutorials when learning how to build a case using the **DivGeo** and **SOLPS-GUI** utilities. Other cases can be added at the suggestion of users, if the data they contain can be made publicly accessible.

For the unstructured code version, the working examples are presently limited to *COMPASS_16515_1150ms_D_AFN*, an unstructured, narrow grid, pure D COMPASS case with drifts and Advanced Fluid Neutrals, and kinetic neutrals without drifts, with a Matlab example script to plot results in the WG code, and *COMPASS_sensitivity_D+T*, a CI test case and demonstration for the sensitivity calculation and optimization toolbox based on Algorithmic Differentiation [24], based on the previous *COMPASS_16515_1150ms_D_AFN* case, but with a coarser mesh, using the PARDISO solver, deuterium + tritium species, drifts, Advanced Fluid Neutrals, the Zhdanov closure and the *k*-model.

The Zenodo records, at least the ones involved in CI testing, are updated whenever a code bug fix or new feature means that the solutions will have evolved from their previous state. The code distribution contains, inside its *runs/examples* directory, references to the corresponding record number with that code distribution commit, so that the matching set of examples will be fetched on demand.

User engagement and community development

The SOLPS-ITER development process follows modern collaborative software-engineering practices. Users may report issues, request features, and participate in technical discussions through GitHub, using dedicated tabs on the repositories' home pages. Users can create forks in which to make changes necessary for their purpose, develop new features, or test bug fixes. Proposed code modifications and/or documentation complements, insofar as they adhere to the contributing guidelines provided with the code distribution, can then be submitted for inclusion into the main code distribution through pull requests, which automatically trigger CI validation tests using ITER Organization computing resources. Documentation previously distributed through internal platforms that required an ITER IDM account has been progressively migrated toward publicly accessible GitHub Wiki pages and MicroSoft (MS) Teams resources. New users are encouraged to make themselves known to the code Responsible Officer at the IO (first author of this paper) to be added to the SOLPS-dedicated MS Teams and Slack channel email distribution lists to stay informed about the status of the code and related events.

This transition substantially simplifies access to the software ecosystem and facilitates interaction among developers and users worldwide, in particular for researchers outside the ITER Member States and/or actors from private fusion ventures.

Future developments

Current development priorities focus on extending the capabilities of the Wide Grids framework to reach and ultimately surpass feature parity with the structured-grid versions. A key ingredient towards this goal is a more versatile grid generation method, that goes beyond the topological limitations of **Carre2** and can easily handle arbitrary topologies. For this purpose, the KU Leuven group has been developing the GOAT (Grid Generation, Optimization and Adaptation Toolbox) [25] framework (available at https://github.com/TME-TFE/goat) and the necessary workflow to fully integrate it within the SOLPS-ITER case build-up chain. Other advancements of the Wide Grid code capabilities already in development, and that can be found in various branches on the SOLPS-ITER and submodule repositories include, among others, new boundary conditions formulations for neutral density and temperature in the AFN model, extending the AFN model to include molecules, a module to model ECWC power sources and glow discharge plasmas [5], photon transport and opacity effects [26], a package for modelling liquid–metal divertors, coupling to wall inventory models, and exploration of additional sparse matrix solvers.

Additional effort is also placed on enhanced coupling to external modelling tools. Ongoing activities include interfaces to IMPGYRO [27], possible coupling to the Eunomia neutral transport code [28], and implementation within integrated-modelling workflows based on IMAS actors [29].

Conclusions

The release of SOLPS-ITER as an open-source software suite represents a significant step toward broader accessibility of advanced edge-plasma modelling tools. The public distribution includes not only the simulation framework itself but also a growing ecosystem of supporting utilities, benchmark datasets, and development infrastructure. The availability of reference cases through Zenodo and collaborative development through GitHub provides a foundation for reproducible

research, transparent software evolution and broader community participation. Future efforts will focus on continued development of the Wide Grids framework together with tighter integration into the IMAS ecosystem and enhanced coupling to external physics models.

CRediT authorship contribution statement

X. Bonnin: Writing – original draft, Validation, Software, Project administration, Methodology, Formal analysis, Data curation, Conceptualization. **O. Hoenen:** Software, Resources, Methodology. **S.D. Pinches:** Supervision, Software, Project administration, Methodology. **A.A. Pshenov:** Writing – review & editing, Validation, Software. **W. Dekeyser:** Software, Resources, Project administration, Methodology, Formal analysis, Conceptualization. **W.Van Uytven:** Writing – review & editing, Validation, Software. **E. Kaveeva:** Writing – review & editing, Supervision, Formal analysis. **N. Shtyrkhunov:** Writing – review & editing, Visualization, Validation, Software, Investigation, Data curation. **A. Zito:** Validation, Software.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The work presented in this paper has been performed in part under the auspices of the ITER Fellowship scheme. The views and opinions expressed therein do not necessarily reflect those of the ITER Organization.

Data availability

The code source is available at https://github.com/iterorganization/SOLPS-ITER and the example run data can be found at https://zenodo.org/communities/solps/ .

References

[1] S. Wiesen, et al., J. Nucl. Mater. 463 (2015) 480–484.
[2] X. Bonnin, et al., Plasma Fusion Res. 11 (2016) 1403102.
[3] W. Dekeyser, et al., Nucl. Mater. Energy 27 (2021) 100999.
[4] M. Sala, et al., Plasma Phys. Control. Fusion 62 (2020) 055005.
[5] A. Piras et al., Contrib. Plasma Phys. (2026), in press.
[6] V. Rozhansky, et al., Nucl. Fusion 49 (2009) 025007.
[7] D. Reiter, et al., Fusion Sci. Technol. 47 (2005) 172–186.
[8] R. Marchand, et al., Comput. Phys. Commun. 96 (1996) 232–246.
[9] L. Kos et al., Proc. NENE 2016, Portorož, Slovenia (2016).
[10] F. Imbeaux, et al., Nucl. Fusion 55 (2015) 123006.
[11] W. Dekeyser, et al., Nucl. Mater. Energy 18 (2019) 125–130.
[12] J.D. Lore, et al., Nucl. Fusion 63 (2023) 046015.
[13] J.-S. Park, et al., Nucl. Fusion 64 (2024) 076036.
[14] A. Zito et al., these proceedings.
[15] F. Cursi et al., Contrib. Plasma Phys. (2026), in press.
[16] V. Korzueva, et al., Nucl. Fusion. 66 (2026) 106027.
[17] M. Carpita, et al., Nucl. Fusion 65 (2025) 116001.
[18] S.O. Makarov, et al., Nucl. Fusion 63 (2023) 026014.
[19] S.O. Makarov et al., Phys. Plasmas 28 (2021) 062308; Erratum Phys. Plasmas 31 (2024) 089903.
[20] S.O. Makarov, et al., Contrib. Plasma Phys. 62 (2022) e202100165.
[21] W. Van Uytven, et al., Nucl. Fusion 62 (2022) 086023.
[22] W. Dekeyser, et al., Contrib. Plasma Phys. 62 (2022) e202100190.
[23] N. Shtyrkhunov, et al., Nucl. Fusion 66 (2026) 056019.
[24] S. Carli, et al., J. Comp. Phys. 491 (2023) 112403.
[25] S. Van den Kerkhof, et al., Contrib, Plasma Phys. 64 (7–8) (2024) e202300134.
[26] R. Chandra et al., these proceedings.
[27] S. Yamoto, et al., Comput. Phys. Commun. 248 (2020) 106979.
[28] R.C. Wieggers, et al., Contrib. Plasma Phys 52 (2012) 440–444.
[29] D. Coster, these proceedings.